

< PYTHON DATA SCIENCE CLASSROOM

Pandas DataFrames

From Python Collections to Real-World Data Analysis

`['Anu', 'Rahul']`

Messy
Collections



`pd.DataFrame()`

Structured Table

Why Pandas?

02 / 45

Standard Python Collections

- ✓ **Lists:** Great for 1D sequences of items.
- ✓ **Tuples:** Fixed, immutable records.
- ✓ **Dictionaries:** Key-value paired lookup tables.

⚠ Keeping multiple parallel lists synchronized manually is error-prone!

Scaling to 10,000+ Observations

Real datasets contain thousands of rows and dozens of attributes.



**"Would you manually manage Lists and Dictionaries
for 10,000 observations?"**

✓ Pandas merges everything into a clean tabular layout!

What is Pandas?

03 / 45



Python Library

An open-source data manipulation library specifically created for analysis and data science in Python.



Tabular Data

Engineered for working effortlessly with structured grid data containing rows and columns.



Fast Analysis

Provides powerful tools to load, clean, filter, aggregate, and analyze high-volume data quickly.

PANDAS CORE ARCHITECTURE

→ Rows + Columns Grid

→ Effortless Analysis

Import Pandas

04 / 45



script.py

```
# Import the library into Python
```

```
import pandas as pd
```

Code Breakdown

- > **pandas:** The full name of the Python library package.
- > **pd:** The standard, universally accepted short alias.

WHY USE 'PD'?

Instead of writing **pandas.DataFrame()** repeatedly, developers write **pd.DataFrame()** for conciseness!

What is a Series?

05 / 45

1D Labeled Array

A **Series** is essentially a **single column** of data in Pandas.

- ✓ Holds a sequence of values.
- ✓ Has an associated **index** (row labels).
- ✓ All elements typically share the same data type.

Index	Series Values (Column)

+ Creating a Series

06 / 45



create_series.py

```
import pandas as pd

# Create a 1D Pandas Series from a Python List
temperatures = pd.Series([28.5, 30.2, 29.1, 31.5, 27.8])
print(temperatures)
```

>_ CONSOLE OUTPUT

```
0    28.5
1    30.2
2    29.1
3    31.5
4    27.8
```

dtype: float64

👉 pandas automatically generates a 0-based numeric index on the left!

Index + Value Structural View

07 / 45

Index (Row Label)	Value (Data)
	28.5
	30.2
	29.1
	31.5
	27.8

Key Takeaway

Every element in a Series consists of two components:

 **Index:** Identifier for retrieving row values.

 **Value:** The actual data stored in the Series.

Unless custom labels are supplied, Pandas defaults to 0, 1, 2, ... N-1.

👉 Accessing Series Values

08 / 45



access_series.py

```
# Retrieve value at index 0
print (temperatures [0])

# Retrieve value at index 3
print (temperatures [3])
```

>_ CONSOLE OUTPUT

28.5

31.5

🔗 **Python List Analogy:** Accessing Pandas Series values uses square bracket index syntax `series[index]`, identical to standard Python lists!

Built-in Series Calculations

09 / 45



stats.py

```
print (temperatures .mea ()) # Average  
print (temperatures .ma ()) # Largest  
print (temperatures .min ()) # Smallest
```

.mean() 29.42 °C

Calculates arithmetic average of column values.

.max() 31.5 °C

Extracts maximum value stored in Series.

.min() 27.8 °C

Extracts minimum value stored in Series.

Series vs DataFrame

10 / 45

1-DIMENSIONAL

Series

A single column of elements with row index labels.

Index	Maths
0	85
1	72

2-DIMENSIONAL

DataFrame

A complete table combining multiple Series side-by-side!

Index	Name	Maths
0	Anu	85
1	Rahul	72

Creating a DataFrame

11 / 45



create_df.py

```
data = {  
    "Name": ["Anu" , "Rahul" , "Meera" ],  
    "Age" : [19, 20, 19],  
    "Maths" : [85, 72, 91],  
    "Physics" : [91, 78, 88]  
}
```

```
df = pd.DataFrame (data )  
print (df )
```

Dict to Table Mapping

In a Python Dictionary:

 **Keys** ("Name", "Age") become **Column Headers**.

 **Lists** become **Column Values**.

DATAFRAME OUTPUT GRID

3 rows × 4 columns successfully generated!

Understanding a DataFrame

12 / 45

Index				
	Name	Age	Maths	Physics
0	Anu	19	85	91
1	Rahul	20	72	78
2	Meera	19	91	88

Index

Row identifiers automatically assigned starting from 0.

Columns

Attributes labeled "Name", "Age", "Maths", "Physics".

Grid Composition

Each individual column is a Series; the whole table is a DataFrame.



Why Explore First?

13 / 45

1. Load Data

`read_csv()`



2. Explore Data

`head() / info()`



3. Understand

check structure



4. Clean Data

`dropna()`



5. Analyze

insights



Golden Rule of Data Science

"Do NOT immediately start calculating. First understand your dataset!"

Exploring with head()

14 / 45



head_demo.py

Default: returns top 5 rows

```
df.head()
```

Specify custom row count

```
df.head(2)
```

What does head() do?

- ✓ Previews the first 5 rows of a large dataset by default.
- ✓ Perfect for verifying column names and data formatting.

DF.HEAD(2) RESULT

Displays only rows 0 and 1 instantly!

i Exploring with info()

15 / 45



info_demo.py

```
df.info()
```

>_ STRUCTURE OVERVIEW

RangeIndex: 3 entries, 0 to 2

Data columns (total 4 columns):

#	Column	Non-Null Count	Dtype
0	Name	3 non-null	object
1	Age	3 non-null	int64
2	Maths	3 non-null	int64
3	Physics	3 non-null	int64

🔍 Key Discoveries from info()

- ✓ **Column Names:** Verifies spelling of headers.
- ✓ **Non-Null Count:** Instantly exposes missing data!
- ✓ **Data Types (dtype):** Shows int64, float64, or object.

Exploring with describe()

16 / 45



describe_demo.py

```
df.describe()
```

	Age	Maths	Physics
count	19.33	82.67	85.67
mean	19.00	72.00	78.00
std	20.00	91.00	91.00

Statistical Summary

Automatically calculates summary statistics for all numerical columns:

count: row total

mean: average

std: spread

min/max: bounds

25%/50%/75%: quartiles

The Essential Exploration Trio

17 / 45



`df.head()`

What does the data look like?

Previews sample rows to inspect actual values visually.



`df.info()`

What is the structure?

Checks column dtypes, row counts, and missing data.



`df.describe()`

What do the numbers say?

Computes statistical summaries across numeric attributes.



Why CSV Format?

18 / 45

Real-World Data Storage

In real data science projects, data is almost never hardcoded manually in code.

- ✓ Exported from databases, sensors, & spreadsheets.
- ✓ Lightweight, plain-text format supported everywhere.



Raw CSV File



Pandas DataFrame Table

What is CSV? (Comma-Separated Values)

19 / 45



students.csv (Plain Text)

```
Name, Age, Maths, Physics  
Anu, 19, 85, 91  
Rahul, 20, 72, 78  
Meera, 19, 91, 88
```

How CSV Translates

- > **First line:** Column headers separated by commas.
- > **Subsequent lines:** Individual data rows.
- > **Commas:** Delimit boundaries between cells.

Import a CSV File

20 / 45



read_data.py

```
import pandas as pd

# Load CSV dataset into DataFrame
df = pd.read_csv("students.csv")

print(df.head())
```



Google Colab Tip

If running code inside Google Colab or Jupyter Notebooks:

Always upload **students.csv** to the notebook session storage before calling **pd.read_csv()**!

🔽 Selecting One Column

21 / 45



select_column.py

```
# Select single column using single bracket
maths_scores = df["Maths"]
print(maths_scores)
```

Resulting Data Structure

RETURNS A SERIES

Selecting a single column using single brackets `df["Maths"]` extracts a 1D **Pandas Series**.

```
0 85
1 72
2 91
Name: Maths, dtype: int64
```

📄 Selecting Multiple Columns

22 / 45



select_multiple.py

```
# Note the DOUBLE brackets! [[ ]]  
subset = df [ ["Name" , "Maths" ] ]  
print (subset )
```

Single vs Double Brackets

`df["Maths"]` → 1D Series

`df[["Name", "Maths"]]` → 2D DataFrame

Why Filter Data?

23 / 45

🎯 Classroom Scenario

Imagine analyzing a dataset with 500 students.

Goal: "Extract rows for students who scored higher than 80 in Maths."

Full Dataset (500 Rows)



Filtered Subset (Matching Criteria)

Filtering with One Condition

24 / 45



filter_one.py

```
# Filter rows where Maths > 80
top_maths = df[df["Maths"] > 80]
print(top_maths)
```

Boolean Mask Under the Hood

`df["Maths"] > 80` creates a True/False mask:

```
0 True (85 > 80) → KEPT
1 False (72 > 80) → REMOVED
2 True (91 > 80) → KEPT
```



Interactive Check: Predict the Filter

25 / 45



quiz.py

```
# Predict which students will appear!  
df [df ["Age" ] ≥ 20]
```

Name	Age
Anu	19
Rahul	20
Meera	19



Class Challenge

Which rows satisfy **Age ≥ 20**?

ANSWER

Only Row 1 (Rahul - Age 20) is returned!

Filtering Multiple Conditions

26 / 45



filter_multi.py

```
# Use & (AND) or | (OR)
# Wrap each condition in parentheses ()
df [(df ["Maths" ] > 80) & (df ["Physics" ] > 80)]
```

⚠ Syntax Warnings

- ✓ Use bitwise operators: **&** (AND), **|** (OR).
- ⚠ Do NOT write Python words **and** / **or**!
- ✓ Always wrap individual condition expressions in parentheses **()**.

Why Clean Data?

27 / 45

Messy Real-World Defects

- ✗ **Missing Values:** Empty cells due to sensor failures.
- ✗ **Duplicates:** Repeated survey entries.
- ✗ **Typos:** Misspelled categories or corrupt numbers.

GARBAGE IN = GARBAGE OUT

Data cleaning transforms untrustworthy raw records into reliable inputs for scientific decision-making.



Identifying Missing Values (NaN)

28 / 45

Name	Age	Maths
Anu	19	85
Rahul	20	
Meera	19	91

? What is NaN?

NaN stands for **Not a Number** in Pandas.

It represents missing, unrecorded, or corrupted data cells in tabular datasets.

Find Missing Values

29 / 45



isnull_demo.py

```
# Returns True/False grid
df.isnull()

# Count missing items per column
df.isnull().sum()
m
```

Output Breakdown

```
Name 0
Age 0
Maths 1
dtype: int64
```

Combining `.isnull().sum()` gives an immediate total count of missing cells per column.

Remove Missing Rows: dropna()

30 / 45



dropna_demo.py

```
# Drop any row containing NaN
clean_df = df.dropna()
print(clean_df)
```

How dropna() Works

Removes the entire row if **any** cell contains a missing **NaN** value.

Rahul's row (which contained NaN) is completely removed!

Fill Missing Values: fillna()

31 / 45



fillna_demo.py

```
# Fill NaN with mean score
mean_maths = df["Maths"].mean()
df["Maths"] = df["Maths"].fillna(mean_maths)
```

Best Practice Note

"Do NOT blindly fill all NaNs with the mean! The right choice depends on domain context."

Name	Maths
Rahul	

Handling Duplicate Rows

32 / 45



duplicates.py

```
# Check for duplicate rows
print (df.duplicated ())

# Drop duplicate rows
df = df.drop_duplicates ()
```

Duplicate Detection

- ✓ `duplicated()` returns a boolean mask indicating identical rows.
- ✓ `drop_duplicates()` purges repeated records automatically.

Complete Pandas Workflow

33 / 45

1. LOAD

`read_csv()`

2. EXPLORE

`head() / info()`

3. SELECT

`df[["Col"]]`

4. FILTER

`df[df>80]`

5. CLEAN

`dropna()`

6. ANALYZE

`mean() / max()`

Core Concepts Summary

34 / 45

Data Structures

Series: 1D labeled column.

DataFrame: 2D structured grid table.

Exploration

head(): Top rows.

info(): Structure overview.

describe(): Numeric stats.

Cleaning & Filter

dropna(): Remove NaNs.

fillna(): Fill NaNs.

df[cond]: Filter subset.

★ CORE PRINCIPLE

LOAD → LOOK → UNDERSTAND → CHECK → CLEAN →
FILTER → ANALYSE

"When you receive a dataset, don't immediately start calculating. First understand your data!"

PART 10 – PRACTICAL LAB

Now Let's Work With Real Scientific Data

Applying all our Pandas skills to analyze a real-world scientific dataset.

Step 1: Load Scientific Dataset

37 / 45



scientific_lab.py

```
import pandas as pd

# Load real scientific observations
df = pd.read_csv("scientific_data.csv")
print(df)
```



Dataset Overview

Importing **scientific_data.csv** creates a Pandas DataFrame containing environmental sensor observations.

Step 2: Explore with head()

38 / 45



explore_head.py

```
df.head()
```

Classroom Question

"Why do we execute head() first?"

Expected Idea: To get an instant visual preview of actual column names and row formatting!

Step 3: Explore with info()

39 / 45



explore_info.py

```
df.info()
```

Structural Findings

- ✓ Confirms non-null count for sensor columns.
- ✓ Verifies floating point numerical types (`float64`).

Step 4: Explore with describe()

40 / 45



explore_desc.py

```
df.describe()
```

head()

→ Appearance

info()

→ Structure

describe()

→ Statistics

🌡️ Step 5: Select Scientific Variable

41 / 45



temp_analysis.py

```
# Extract Temperature column
temp = df["Temperature"]

print(temp.agg(['mean', 'max', 'min']))
```

📊 Temperature Metrics

Extracting a single Series allows calculating summary environmental metrics easily.

? Step 6: Ask a Scientific Question

42 / 45



query.py

```
# Question: Which observations had Temp > 30°C?  
hot_days = df[df["Temperature"] > 30]  
print (hot_days)
```

🔹 Filtering in Science

Reuses our boolean mask filtering concept to isolate extreme weather observations instantly.

Step 7: Validate Sensor Data Integrity

43 / 45



validate.py

```
# Check missing sensor records
```

```
df.isnull().sum()  
m
```

✔ Before Trusting Findings

Always verify missing counts before publishing scientific reports. Apply `dropna()` or `fillna()` if needed!

1. LOAD

`read_csv()`

2. EXPLORE

`head()` / `info()` / `describe()`

3. ANALYZE

Select & Filter

4. CHECK

`isnull().sum()`

 CONGRATULATIONS!

REAL DATASET → LOAD → EXPLORE → CHECK →
ANALYZE

"And that's the complete workflow for analyzing scientific datasets in
Pandas!"